

MATH00041: Introduction to Programming

Randomness - Pandas

Lecture 11

Archontis Giannakidis, PhD

Outline

- **Randomness**
- **Pandas library**

NumPy Random

Generate Random Number

NumPy offers the random module to work with random numbers.

Example:

Generate a random integer from 0 to 100:

```
from numpy import random
```

```
x = random.randint(100)
```

```
print(x)
```

```
>>>
```

```
87
```

Example:

Generate a random float from 0 to 1:

```
from numpy import random
```

```
x = random.rand()
```

```
print(x)
```

```
>>>
```

```
0.37520466792563156
```

In NumPy we work with arrays, and you can use the two methods from the above examples to make random arrays.

Example:

Generate a 1-D array containing 5 random integers from 0 to 100:

```
from numpy import random
```

```
x=random.randint(100, size=(5))
```

```
print(x)
```

```
>>>
```

```
[92 97 1 93 46]
```

np.rand

A variety of functions are accessible within NumPy's random module to create ndarrays initialised with random values. For instance, a 3x4 matrix initialised with random floats between 0 and 1 (uniform distribution) can be created as follows:

Example:

Create an array using rand function:

```
import numpy as np  
arr = np.random.rand(3,4)
```

```
print(arr)
```

```
>>array([[0.31506811, 0.9962061 , 0.22576662, 0.90354811],  
        [0.75222452, 0.76662349, 0.1099069 , 0.98423873],  
        [0.83335318, 0.35623138, 0.67330209, 0.23273007]])
```

Combine Color Size and Alpha

You can combine a colormap with different sizes of the dots. This is best visualized if the dots are transparent:

Example:

Create random arrays with 100 values for x-points, y-points, colors and sizes:

```
import matplotlib.pyplot as plt
import numpy as np
```

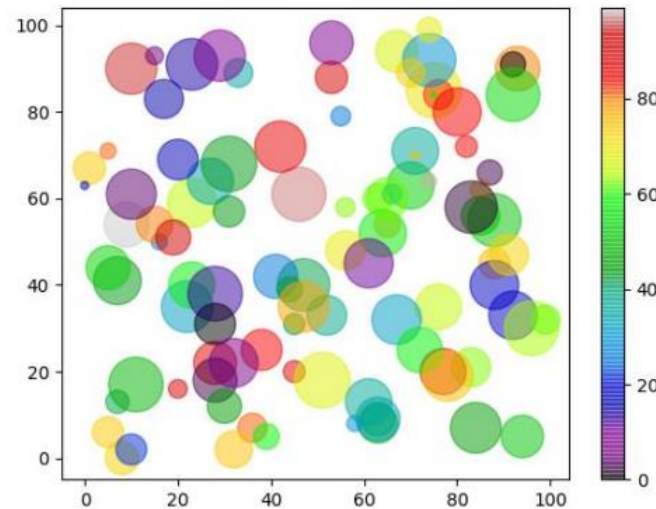
```
x = np.random.randint(100, size=(100))
y = np.random.randint(100, size=(100))
colors = np.random.randint(100, size=(100))
sizes = 10 * np.random.randint(100, size=(100))
```

```
plt.scatter(x, y, c=colors, s=sizes, alpha=0.5, cmap='nipy_spectral')
```

```
plt.colorbar()
```

```
plt.show()
```

```
>>
```



What is Data Distribution?

Data distribution is a list of all possible values and their corresponding frequencies. These lists are crucial when working with statistics and data science. The random module offers methods that return randomly generated data distributions.

Random Distribution

A random distribution is a set of random numbers that follow a certain probability density function.

Probability Density Function: A function that describes a continuous probability. i.e. probability of all values in an array.

Generating random numbers based on defined probabilities is made possible using the `choice()` method from the `random` module.

This method allows us to specify the probability for each value, which can be set as a number between 0 and 1. A probability of 0 indicates that the value will never occur, while a probability of 1 indicates that the value will always occur.

Example:

Generate a 1-D array containing 100 values, where each value has to be 3, 5, 7 or 9.

The probability for the value to be 3 is set to be 0.1

The probability for the value to be 5 is set to be 0.3

The probability for the value to be 7 is set to be 0.6

The probability for the value to be 9 is set to be 0

```
from numpy import random
```

```
x = random.choice([3, 5, 7, 9], p=[0.1, 0.3, 0.6, 0.0], size=(100))
```

```
print(x)
```

```
>>>
```

```
[3 7 7 5 7 7 5 7 7 7 5 5 7 7 7 5 7 5 7 7 5 7 5 5 7 7 7 7
5 7 7 3 5 7 5 7 7 7 7 7 7 5 3 5 7 7 7 7 3 7 5 7 5 7 5 7 7 5
7 7 5 5 7 5 3 5 5 7 7 7 3 7 7 7 5 5 7 7 5 7 7 7 7 7 7 3 7 7
7 7 7 7 7 3 5 3 7 7 7]
```

The sum of all probability numbers should be 1. Even if you run the example above 100 times, the value 9 will never occur. You can return arrays of any shape and size by specifying the shape in the size parameter.

Example:

Same example as above but return a 2-D array with 3 rows, each containing 5 values.

```
from numpy import random
```

```
x = random.choice([3, 5, 7, 9], p=[0.1, 0.3, 0.6, 0.0], size=(3, 5))
```

```
print(x)
```

```
>>>
```

```
[[7 7 7 5 7]
 [7 7 5 7 7]
 [7 7 7 3 7]]
```

Seaborn

Seaborn is a library that uses Matplotlib underneath to plot graphs. It will be used to visualize random distributions. **Distplot** stands for distribution plot, it takes as input an array and plots a curve corresponding to the distribution of points in the array.

Import the Seaborn module in your code using the following statement:

```
import seaborn as sns
```

Normal (Gaussian) Distribution

The Normal Distribution, also known as the Gaussian Distribution, named after German mathematician Carl Friedrich Gauss, is one of the most significant probability distributions. It is a good fit for many events, such as IQ scores, heartbeat, and others. To obtain a Normal Data Distribution, use the `random.normal()` method, which has three parameters.

loc - (Mean) where the peak of the bell exists.

scale - (Standard Deviation) how flat the graph distribution should be.

size - The shape of the returned array.

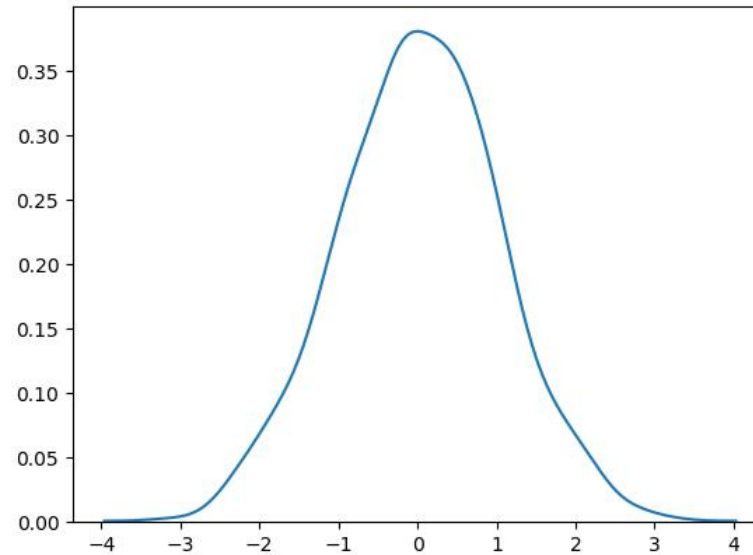
Example:

```
from numpy import random
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.distplot(random.normal(size=1000), hist=False)
```

```
plt.show()
```

```
>>>
```



- ❖ **Note:** The Normal Distribution curve is also known as the Bell Curve because of the bell-shaped curve.

Following is a 3x4 matrix containing random floats sampled from a univariate normal distribution (Gaussian distribution) of mean 0 and variance 1:

Example:

Create an array using rand function:

```
import numpy as np  
arr = np.random.randn(3,4)
```

```
print(arr)
```

```
>>array([[ 1.56110261e+00,  2.43164383e-01, -7.35871062e-01,  
          3.70101675e-04],  
 [ 1.86890564e-01,  6.30621790e-01,  7.30944692e-01,  
        -2.12995795e+00],  
 [-5.61847627e-01, -8.63385596e-01, -3.25545246e-01,  
          7.48875396e-01]])
```

Uniform Distribution

Random events have equal probability, such as generating random numbers.

It has three parameters:

a - lower bound - default 0 .0.

b - upper bound - default 1.0.

size - The shape of the returned array.

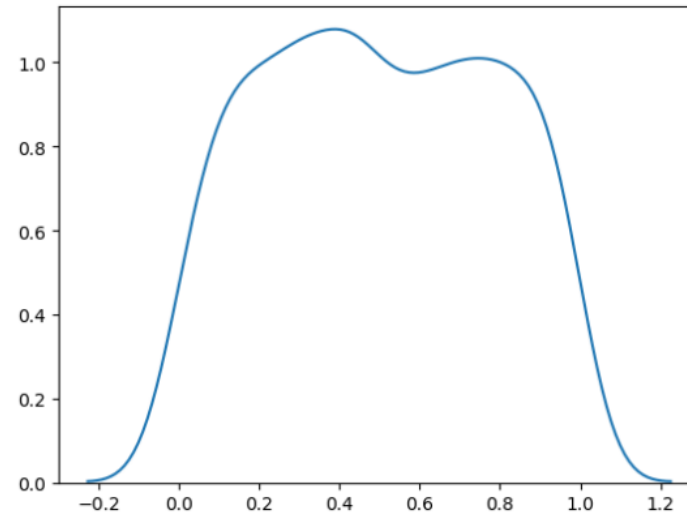
Example:

```
from numpy import random
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.distplot(random.uniform(size=1000), hist=False)
```

```
plt.show()
```

```
>>>
```



Matplotlib Histograms

A histogram is a type of graph that displays the frequency distribution of a dataset. It shows the number of observations falling within each interval or range of values.

For instance, if you collect data on the heights of 250 individuals, a histogram can be created to show the distribution of heights. This type of graph provides a visual representation of the frequency of occurrences for each height range, making it easier to observe patterns and trends within the data.

Create Histogram

To create histograms in Matplotlib, we can utilize the `hist()` function. This function requires an array of numbers as input to generate the histogram. In this example, we will use NumPy to generate an array with 250 values, centered around 170, and with a standard deviation of 10 for simplicity.

The `hist()` function will read the array and produce a histogram:

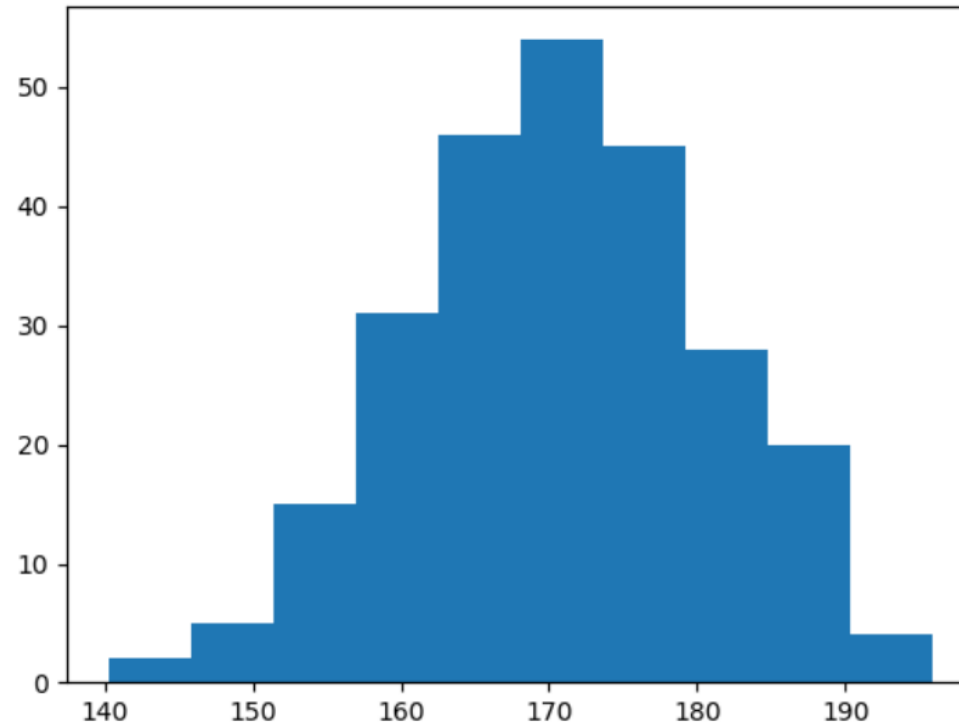
Example:

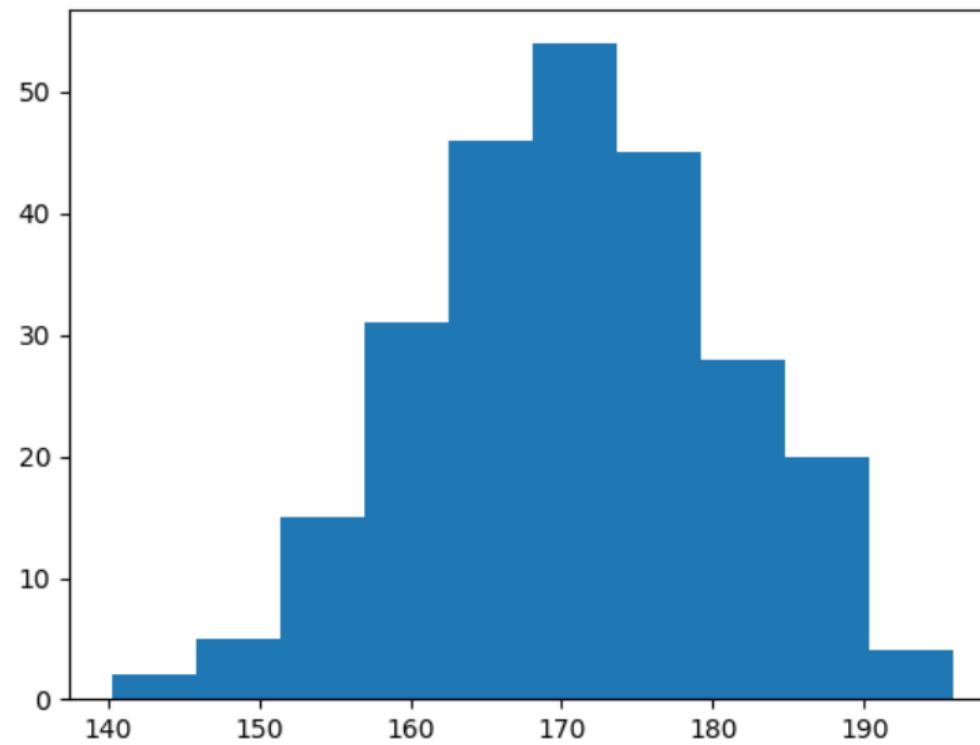
A simple histogram:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x)
plt.show()
>>
```





You can read from the histogram that there are approximately:

- 2 people from 140 to 145cm
- 5 people from 145 to 150cm
- 15 people from 151 to 156cm
- 31 people from 157 to 162cm
- 46 people from 163 to 168cm
- 53 people from 168 to 173cm
- 45 people from 173 to 178cm
- 28 people from 179 to 184cm
- 21 people from 185 to 190cm
- 4 people from 190 to 195cm

Pandas

What is Pandas?

Pandas is a Python library that is mainly used for working with data sets. This powerful library provides a wide range of functions for analysing, cleaning, exploring, and manipulating data. The name "Pandas" is a combination of "Panel Data" and "Python Data Analysis" and it was created by Wes McKinney in 2008. With its easy-to-use and efficient tools for data analysis, Pandas has become an essential tool for data scientists, researchers, and analysts.

Why Use Pandas?

Pandas enable the analysis of large datasets and draw statistical conclusions. It can also clean, messy data and make it readable and relevant. Relevant data is crucial in data science.

What Can Pandas Do?

Pandas give you answers about the data. Like:

- Is there a correlation between two or more columns?
- What is the average value?
- Max value?
- Min value?

Pandas can delete irrelevant rows with empty or NULL values, known as data cleaning.

The source code for Pandas is located at this GitHub repository <https://github.com/pandas-dev/pandas>

Installation of Pandas

```
C:\Users\Your Name>pip install pandas
```

Import Pandas

Once Pandas is installed, import it in your applications by adding the **import** keyword:

```
import pandas
```

Pandas as pd

Pandas is commonly imported as pd alias in Python. An alias is an alternate name for the same thing. Use the as keyword to create an alias while importing:

```
import pandas as pd
```

Now the Pandas package can be referred to as **pd** instead of **pandas**.

Example:

```
import pandas as pd
```

```
mydataset = {  
    'cars': ["BMW", "Volvo", "Ford"],  
    'passings': [3, 7, 2]}  
myvar = pd.DataFrame(mydataset)
```

```
print(myvar)
```

```
>>      cars  passings  
0      BMW          3  
1  Volvo          7  
2   Ford          2
```

Pandas Series

What is a Series?

A Pandas Series is a data structure in Python's Pandas library that stores a one-dimensional array of data of any type. It is often used to represent a column in a table or a data frame. It is a powerful tool for data manipulation and analysis.

Example:

Create a simple Pandas Series from a list:

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a)
```

```
print(myvar)
```

```
>>  
0    1  
1    7  
2    2  
dtype: int64
```

Labels

In pandas, labels are used to identify and access specific rows and columns of a DataFrame. They can be either numerical or string-based and are used in conjunction with indexing to extract and manipulate data from the DataFrame. Labels can be assigned to both rows and columns and are often used to make the data more easily understandable and accessible. Additionally, labels can be used to rename columns or rows in a DataFrame, making it easier to work with and analyse the data.

Example:

Create a simple Pandas Series from a list:

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a)
```

```
print(myvar[0])
```

```
>>
```

```
1
```

Create Labels

With the **index** argument, you can name your own labels.

Example:

Create your own labels:

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a, index = ["x", "y", "z"])
```

```
print(myvar)
```

```
>>>  
x    1  
y    7  
z    2  
dtype: int64
```

You can access an item by referring to the label when you have created labels.

Example:

Return the value of "y":

```
import pandas
```

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a, index = ["x", "y", "z"])
```

```
print(myvar["y"])
```

```
>>
```

```
7
```

Key/Value Objects as Series

You can also use a key/value object, like a dictionary, when creating a Series.

Example:

Create a simple Pandas Series from a dictionary:

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

```
>>
day1    420
day2    380
day3    390
dtype: int64
```

Note: The keys of the dictionary become the labels.

To select only some of the items in the dictionary, use the **index** argument and specify only the items you want to include in the Series.

Example:

Create a Series using only data from "day1" and "day2":

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories, index = ["day1", "day2"])

print(myvar)
```

```
>>
day1    420
day2    380
dtype: int64
```

DataFrames

DataFrames in Pandas are multi-dimensional tables, while Series represents a column.

Example:

Create a DataFrame from two Series:

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```

```
>>
```

	calories	duration
0	420	50
1	380	40
2	390	45

What is a DataFrame?

A Pandas DataFrame is a two-dimensional labeled data structure with columns of potentially different types, like an Excel spreadsheet or SQL table.

Example:

Create a simple Pandas DataFrame:

```
import pandas as pd
```

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

#load data into a DataFrame object:

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
>>    calories  duration  
0         420         50  
1         380         40  
2         390         45
```

Locate Row

As shown in the above result, a Pandas DataFrame resembles a table with rows and columns. The 'loc' attribute is used to retrieve one or more specific row(s).

Example:

Return row 0:

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

#load data into a DataFrame object:
df = pd.DataFrame(data)

#refer to the row index:
print(df.loc[0])
```

```
>>calories    420
    duration    50
    Name: 0, dtype: int64
```

❖ **Note:** This example returns a Pandas Series.

Example:

Return row 0 and 1:

```
import pandas as pd
```

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
#load data into a DataFrame object:  
df = pd.DataFrame(data)
```

```
#refer to the row index:  
print(df.loc[0:1])
```

```
>>      calories  duration  
0         420         50  
1         380         40
```

❖ **Note:** When using [], the result is a Pandas DataFrame.

Named Indexes

With the `index` argument, you can name your own indexes.

Example:

Add a list of names to give each row a name:

```
import pandas as pd
```

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])
```

```
print(df)
```

```
>>  
      calories  duration  
day1         420         50  
day2         380         40  
day3         390         45
```

Locate Named Indexes

Use the named index in the `loc` attribute to return the specified row(s).

Example:

Return "day2":

```
import pandas as pd
```

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])
```

```
#refer to the named index:
```

```
print(df.loc["day2"])
```

```
>>  
calories    380  
duration     40  
Name: day2, dtype: int64
```

Load Files Into a DataFrame

Pandas is a popular data manipulation library in Python that allows users to work with structured data in a tabular form. If you have data stored in a file, such as a CSV or Excel file, Pandas makes it easy to read and load the data into a DataFrame object, which you can then use to perform various data analysis tasks. This is a common and useful functionality provided by Pandas, especially for data scientists and analysts who work with large datasets.

Example:

Load a comma separated file (CSV file) into a DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df)
```

```
>>      Duration  Pulse  Maxpulse  Calories
0           60    110      130      409.1
1           60    117      145      479.0
2           60    103      135      340.0
3           45    109      175      282.4
4           45    117      148      406.0
..          ...     ...       ...       ...
164          60    105      140      290.8
165          60    110      145      300.4
166          60    115      145      310.2
167          75    120      150      320.4
168          75    125      150      330.4

[169 rows x 4 columns]
```

Read CSV Files

Storing large datasets can be made easy by using CSV files, also known as comma-separated files. These files contain plain text and are a widely used and recognized format that can be read by everyone, including Pandas.

For our examples, we will be working with a CSV file called ['data.csv'](#). You can either download this file or [open it directly](#).

Example:

Load the CSV into a DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())
```

```
>>      Duration  Pulse  Maxpulse  Calories
0           60    110       130      409.1
1           60    117       145      479.0
2           60    103       135      340.0
3           45    109       175      282.4
4           45    117       148      406.0
..          ...     ...       ...       ...
```

If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows:

Example:

Print the DataFrame without the `to_string()` method:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df)
```

```
>>      Duration  Pulse  Maxpulse  Calories
0           60    110      130      409.1
1           60    117      145      479.0
2           60    103      135      340.0
3           45    109      175      282.4
4           45    117      148      406.0
..          ...     ...      ...        ...
164          60    105      140      290.8
165          60    110      145      300.4
166          60    115      145      310.2
167          75    120      150      320.4
168          75    125      150      330.4
```

```
[169 rows x 4 columns]
```

max_rows

The maximum number of rows returned can be checked using the `pd.options.display.max_rows` statement in Pandas option settings.

Example:

Check the number of maximum returned rows:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(pd.options.display.max_rows)
```

```
>> 60
```

To clarify, in my system, there is a default maximum rows number of 60. This means that if a DataFrame contains more than 60 rows, the output of "print(df)" will only include the headers as well as the first and last 5 rows.

Read JSON

Large datasets are frequently saved or retrieved as JSON. JSON is plain text that has an object format and is widely used in the programming world, including Pandas. For our illustrations, we will utilize a JSON file named ['data.json'](#).

Example:

Load the JSON file into a DataFrame:

```
import pandas as pd

df = pd.read_json('data.json')

print(df.to_string())
```

```
>>      Duration  Pulse  Maxpulse  Calories
0           60    110      130      409.1
1           60    117      145      479.0
2           60    103      135      340.0
3           45    109      175      282.4
4           45    117      148      406.0
..          ...     ...     ...     ...
```

Dictionary as JSON

If your JSON code is not in a file, but in a Python Dictionary, you can load it into a DataFrame directly:

Example:

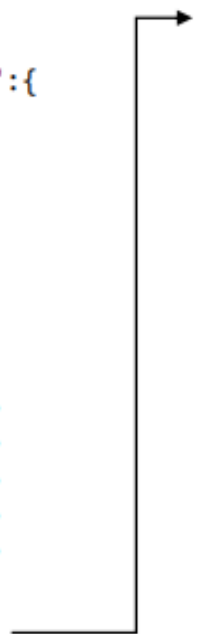
Load a Python Dictionary into a DataFrame:

```
import pandas as pd

data = {
    "Duration":{
        "0":60,
        "1":60,
        "2":60,
        "3":45,
        "4":45,
        "5":60
    },
    "Pulse":{
        "0":110,
        "1":117,
        "2":103,
        "3":109,
        "4":117,
        "5":102
    },
    "Maxpulse":{
        "0":130,
        "1":145,
        "2":135,
        "3":175,
        "4":148,
        "5":127
    },
    "Calories":{
        "0":409,
        "1":479,
        "2":340,
        "3":282,
        "4":406,
        "5":300
    }
}

df = pd.DataFrame(data)

print(df)
```



```
>> Duration  Pulse  Maxpulse  Calories
0         60    110        130     409.1
1         60    117        145     479.0
2         60    103        135     340.0
3         45    109        175     282.4
4         45    117        148     406.0
5         60    102        127     300.5
```

Viewing the Data

One of the most used method for getting a quick overview of the DataFrame, is the `head()` method. The `head()` method returns the headers and a specified number of rows, starting from the top.

Example:

Get a quick overview by printing the first 10 rows of the DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```

```
>>   Duration  Pulse  Maxpulse  Calories
0         60    110       130     409.1
1         60    117       145     479.0
2         60    103       135     340.0
3         45    109       175     282.4
4         45    117       148     406.0
5         60    102       127     300.5
6         60    110       136     374.0
7         45    104       134     253.3
8         30    109       133     195.1
9         60     98       124     269.0
```

In our examples, we will use a CSV file called ['data.csv'](#).

There is also a `tail()` method for viewing the last rows of the DataFrame. The `tail()` method returns the headers and a specified number of rows, starting from the bottom.

Example:

Get a quick overview by printing the first 10 rows of the DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.tail(5))
```

```
>>
      Duration  Pulse  Maxpulse  Calories
164         60    105      140      290.8
165         60    110      145      300.4
166         60    115      145      310.2
167         75    120      150      320.4
168         75    125      150      330.4
```

Info About the Data

The DataFrames object has a method called `info()`, that gives you more information about the data set.

Example:

Get a quick overview by printing the first 10 rows of the DataFrame:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df.info())
```

```
>>>
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Duration    169 non-null    int64
 1   Pulse       169 non-null    int64
 2   Maxpulse    169 non-null    int64
 3   Calories    164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Result Explained

The result tells us there are 169 rows and 4 columns:

```
RangeIndex: 169 entries, 0 to 168  
Data columns (total 4 columns) :
```

And the name of each column, with the data type:

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Duration	169 non-null	int64
1	Pulse	169 non-null	int64
2	Maxpulse	169 non-null	int64
3	Calories	164 non-null	float64

Null Values

The `info()` method provides information on the number of Non-Null values in each column of the dataset. In our data set, the "Calories" column has 164 Non-Null values out of 169, indicating that there are 5 rows with no value for this column. It is essential to note that empty or Null values can negatively impact data analysis. Therefore, it's advisable to remove rows with empty values to ensure data cleanliness.

Data Cleaning

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

In this tutorial, you will learn how to deal with all of them.

Our Data Set

In the next chapters we will use this data set:

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	2020/12/26	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

The data set contains some empty cells ("Date" in row 22, and "Calories" in rows 18 and 28). The data set contains the wrong format ("Date" in row 26). The data set contains the wrong data ("Duration" in row 7). The data set contains duplicates (rows 11 and 12).

Empty Cells

Empty cells can potentially give you a wrong result when you analyse data.

Remove Rows

Dealing with empty cells in a dataset can be done by removing the rows that contain them. This is a viable solution, especially when dealing with large data sets, as removing a few rows will not significantly affect the final outcome.

Example:

Return a new Data Frame with no empty cells:

```
import pandas as pd
df = pd.read_csv('data1.csv')
```

```
new_df = df.dropna()
print(new_df.to_string())
```

```
>>
   Duration      Date  Pulse  Maxpulse  Calories
0         60 '2020/12/01'   110      130     409.1
1         60 '2020/12/02'   117      145     479.0
2         60 '2020/12/03'   103      135     340.0
3         45 '2020/12/04'   109      175     282.4
4         45 '2020/12/05'   117      148     406.0
5         60 '2020/12/06'   102      127     300.0
6         60 '2020/12/07'   110      136     374.0
7        450 '2020/12/08'   104      134     253.3
8         30 '2020/12/09'   109      133     195.1
9         60 '2020/12/10'    98      124     269.0
10        60 '2020/12/11'   103      147     329.3
11        60 '2020/12/12'   100      120     250.7
12        60 '2020/12/12'   100      120     250.7
13        60 '2020/12/13'   106      128     345.3
14        60 '2020/12/14'   104      132     379.3
15        60 '2020/12/15'    98      123     275.0
16        60 '2020/12/16'    98      120     215.2
17        60 '2020/12/17'   100      120     300.0
19        60 '2020/12/19'   103      123     323.0
20        45 '2020/12/20'    97      125     243.0
21        60 '2020/12/21'   108      131     364.2
23        60 '2020/12/23'   130      101     300.0
24        45 '2020/12/24'   105      132     246.0
25        60 '2020/12/25'   102      126     334.5
26        60 '2020/12/26'   100      120     250.0
27        60 '2020/12/27'    92      118     241.0
29        60 '2020/12/29'   100      132     280.0
30        60 '2020/12/30'   102      129     380.3
31        60 '2020/12/31'    92      115     243.0
```

If you want to change the original DataFrame, use the `inplace = True` argument:

Example:

Remove all rows with NULL values:

```
import pandas as pd

df = pd.read_csv('data1.csv')

df.dropna(inplace = True)

print(df.to_string())
```

- ❖ **Note:** Now, the `dropna(inplace = True)` will NOT return a new DataFrame, but it will remove all rows containing NULL values from the original DataFrame.

Replace Empty Values

If you are struggling with empty cells, there is an alternative option to deleting entire rows. Instead, you can insert a new value. The `fillna()` method is an effective way of replacing empty cells with a value.

Example:

Replace NULL values with the number 130:

```
import pandas as pd

df = pd.read_csv('data1.csv')

df.fillna(130, inplace = True)
print(df.to_string())
```

Replace Only For Specified Columns

Example:

Replace NULL values in the "Calories" columns with the number 130:

```
import pandas as pd

df = pd.read_csv('data1.csv')

df["Calories"].fillna(130, inplace = True)

print(df.to_string())
```

The code above demonstrates how to replace empty cells in a specific column of a DataFrame.

Removing Rows

Example:

Remove rows with a NULL value in the "Date" column:

```
import pandas as pd
df = pd.read_csv('data1.csv')
df['Date'] = pd.to_datetime(df['Date'])
df.dropna(subset=['Date'], inplace = True)

print(df.to_string())
```

we can remove the row by using the `dropna()` method.

Replacing Values

One way to fix wrong values is to replace them with something else.

In our example, it is most likely a typo, and the value should be "45" instead of "450", and we could just insert "45" in row 7:

Example:

Set "Duration" = 45 in row 7:

```
import pandas as pd
df = pd.read_csv('data1.csv')

df.loc[7, 'Duration'] = 45

print(df.to_string())
```

For small data sets you might be able to replace the wrong data one by one, but not for big data sets.

To replace wrong data for larger data sets you can create some rules, e.g. set some boundaries for legal values, and replace any values that are outside of the boundaries.

Example:

Loop through all values in the "Duration" column. If the value is higher than 120, set it to 120:

```
import pandas as pd
df = pd.read_csv('data1.csv')

for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.loc[x, "Duration"] = 120
print(df.to_string())
```

Removing Rows

Another way of handling wrong data is to remove the rows that contains wrong data.

This way you do not have to find out what to replace them with, and there is a good chance you do not need them to do your analyses.

Example:

Delete rows where "Duration" is higher than 120:

```
import pandas as pd
df = pd.read_csv('data1.csv')

for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.drop(x, inplace = True)
print(df.to_string())
```

Discovering Duplicates

Duplicate rows are rows that have been registered more than one time.

By looking at our test data set, we can assume that rows 11 and 12 are duplicates.

To discover duplicates, we can use the `duplicated()` method. The `duplicated()` method returns a Boolean value for each row:

Example:

Returns True for every row that is a duplicate, otherwise False:

```
import pandas as pd
df = pd.read_csv('data1.csv')
df.drop_duplicates(inplace = True)
print(df.duplicated())
```

Finding Relationships

The Pandas module comes with a useful feature known as the `corr()` method. This method helps you calculate the relationship between columns in your dataset. To use this method, you need to have a CSV file named ['data2.csv'](#).

Example:

Show the relationship between the columns:

```
import pandas as pd
df = pd.read_csv('data2.csv')
print(df.corr())
```

	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922717
Pulse	-0.155408	1.000000	0.786535	0.025121
Maxpulse	0.009403	0.786535	1.000000	0.203813
Calories	0.922717	0.025121	0.203813	1.000000

❖ **Note:** The `corr()` method ignores "not numeric" columns.

Result Explained

The result of the `corr()` method is a table with many numbers that represents how well the relationship is between two columns.

The number varies from -1 to 1.

1 means that there is a 1 to 1 relationship (a perfect correlation), and for this data set, each time a value went up in the first column, the other one also went up.

0.9 is also a good relationship; if you increase one value, the other will probably increase.

-0.9 would be just as good a relationship as 0.9, but if you increase one value, the other will drop.

0.2 means NOT a good relationship, meaning that if one value goes up does not mean that the other will.

What is a good correlation? It depends on the use, but I think it is safe to say you have to have at least 0.6 (or -0.6) to call it a good correlation.

Perfect Correlation:

We can see that "Duration" and "Duration" got the number 1.000000, which makes sense; each column always has a perfect relationship with itself.

Good Correlation:

"Duration" and "Calories" got a 0.922721 correlation, which is a very good correlation, and we can predict that the longer you work out, the more calories you burn, and the other way around: if you burned a lot of calories, you probably had a long work out.

Bad Correlation:

"Duration" and "Maxpulse" got a 0.009403 correlation, which is a very bad correlation, meaning that we cannot predict the max pulse by just looking at the duration of the workout, and vice versa.

Pandas - Plotting

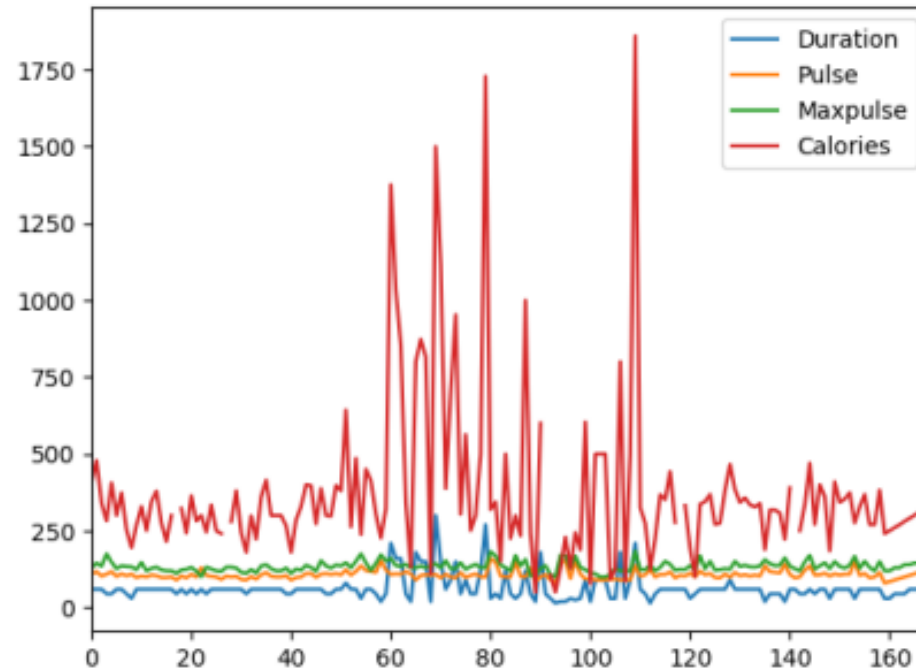
Pandas uses the `plot()` method to create diagrams.

We can use Pyplot, a submodule of the Matplotlib library to visualize the diagram on the screen.

Example:

Import pyplot from Matplotlib and visualize our DataFrame:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data3.csv')
df.plot()
plt.show()
>>
```



Scatter Plot

Specify that you want a scatter plot with the `kind` argument:

```
kind = 'scatter'
```

A scatter plot needs an x- and a y-axis.

In the example below we will use "Duration" for the x-axis and "Calories" for the y-axis.

Include the x and y arguments like this:

```
x = 'Duration', y = 'Calories'
```

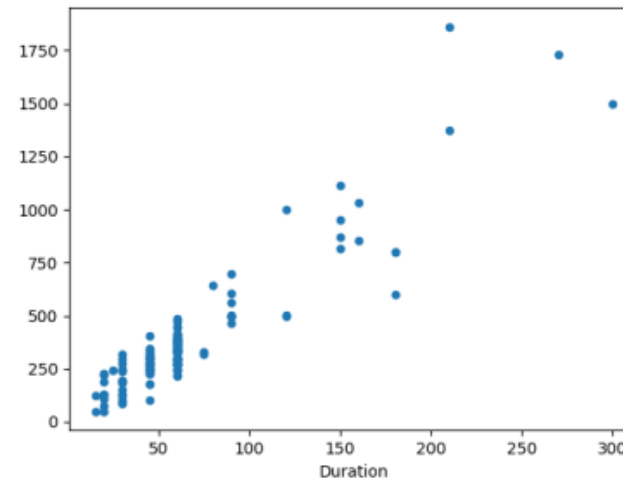
Example:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data3.csv')

df.plot(kind = 'scatter', x = 'Duration', y = 'Calories')

plt.show()
>>
```



Remember: In the previous example, we learned that the correlation between "Duration" and "Calories" was 0.922721, and we concluded with the fact that higher duration means more calories burned. By looking at the scatterplot, I will agree.

Let's create another scatterplot, where there is a bad relationship between the columns, like "Duration" and "Maxpulse", with the correlation 0.009403:

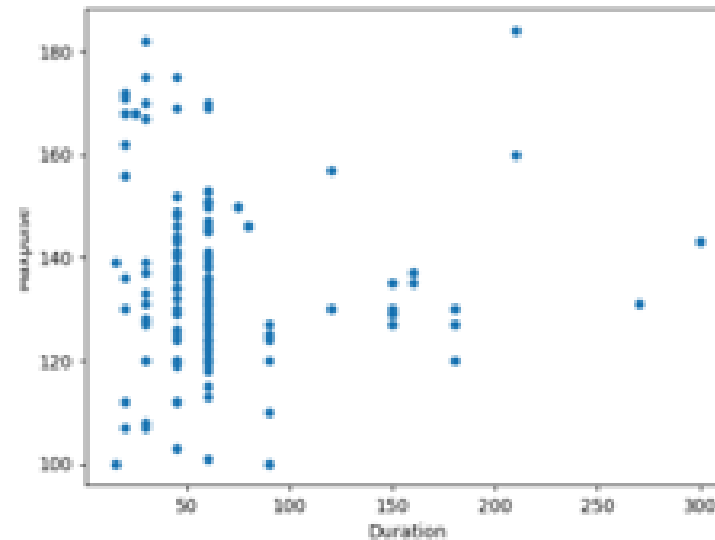
Example:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data3.csv')

df.plot(kind = 'scatter', x = 'Duration', y = 'Maxpulse ')

plt.show()
>>
```



Histogram

Use the `kind` argument to specify that you want a histogram:

```
kind = 'hist'
```

A histogram needs only one column. A histogram shows us the frequency of each interval, e.g. how many workouts lasted between 50 and 60 minutes? In the example below we will use the "Duration" column to create the histogram:

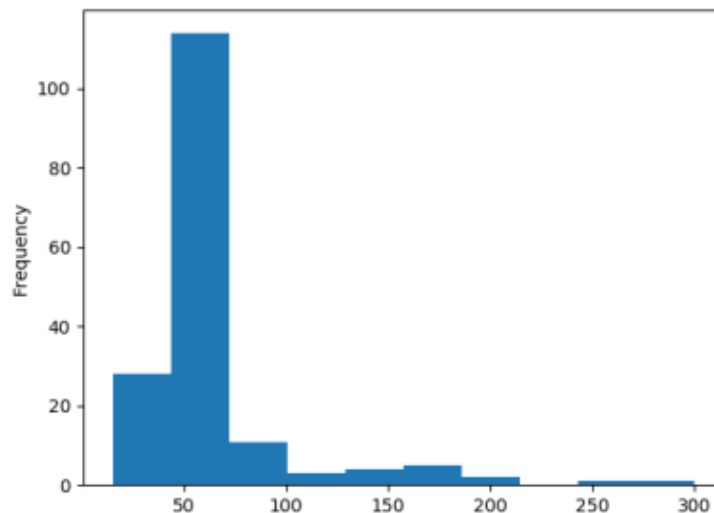
Example:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data3.csv')

df["Duration"].plot(kind = 'hist')

plt.show()
>>
```



❖ **Note:** The histogram tells us that there were over 100 workouts that lasted between 50 and 60 minutes.

Thank you